

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression
x = np.array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9]).reshape(-1, 1)
y = np.array([1, 3, 2, 5, 7, 8, 8, 9, 10, 12])

print("Data loaded successfully!")
```

Data loaded successfully!

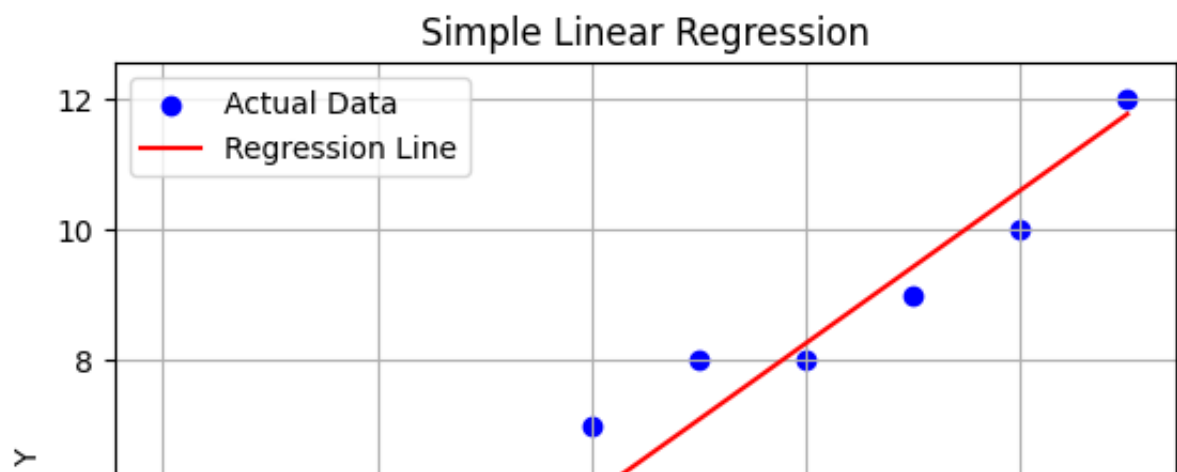
```
model = LinearRegression()
model.fit(x, y)

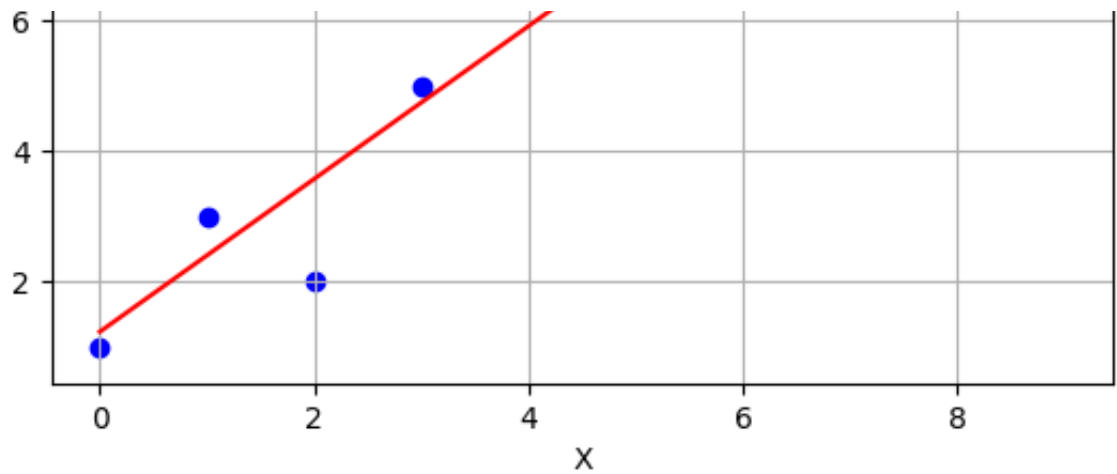
intercept = model.intercept_
slope = model.coef_[0]
print(f"Intercept (Beta 0): {intercept:.2f}")
print(f"Slope (Beta 1): {slope:.2f}")

y_pred = model.predict(x)
plt.scatter(x, y, color='blue', label='Actual Data')
plt.plot(x, y_pred, color='red', label='Regression Line')
plt.title('Simple Linear Regression')
plt.xlabel('X')
plt.ylabel('Y')
plt.legend()
plt.grid(True)
plt.show()

r_sq = model.score(x, y)
print(f"Coefficient of determination (R^2): {r_sq:.4f}")
print("If R^2 is close to 1, the model is highly satisfactory.")
```

Intercept (Beta 0): 1.24
Slope (Beta 1): 1.17





Coefficient of determination (R^2): 0.9525

If R^2 is close to 1, the model is highly satisfactory.

```
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.datasets import make_classification
from sklearn.metrics import accuracy_score, confusion_matrix

X_log, y_log = make_classification(n_samples=100, n_features=2, n_re
                                  n_classes=2, random_state=42)

X_train, X_test, y_train, y_test = train_test_split(X_log, y_log, te
print("Training and testing data prepared!")
```

Training and testing data prepared!

```
log_model = LogisticRegression()
log_model.fit(X_train, y_train)

y_pred_log = log_model.predict(X_test)

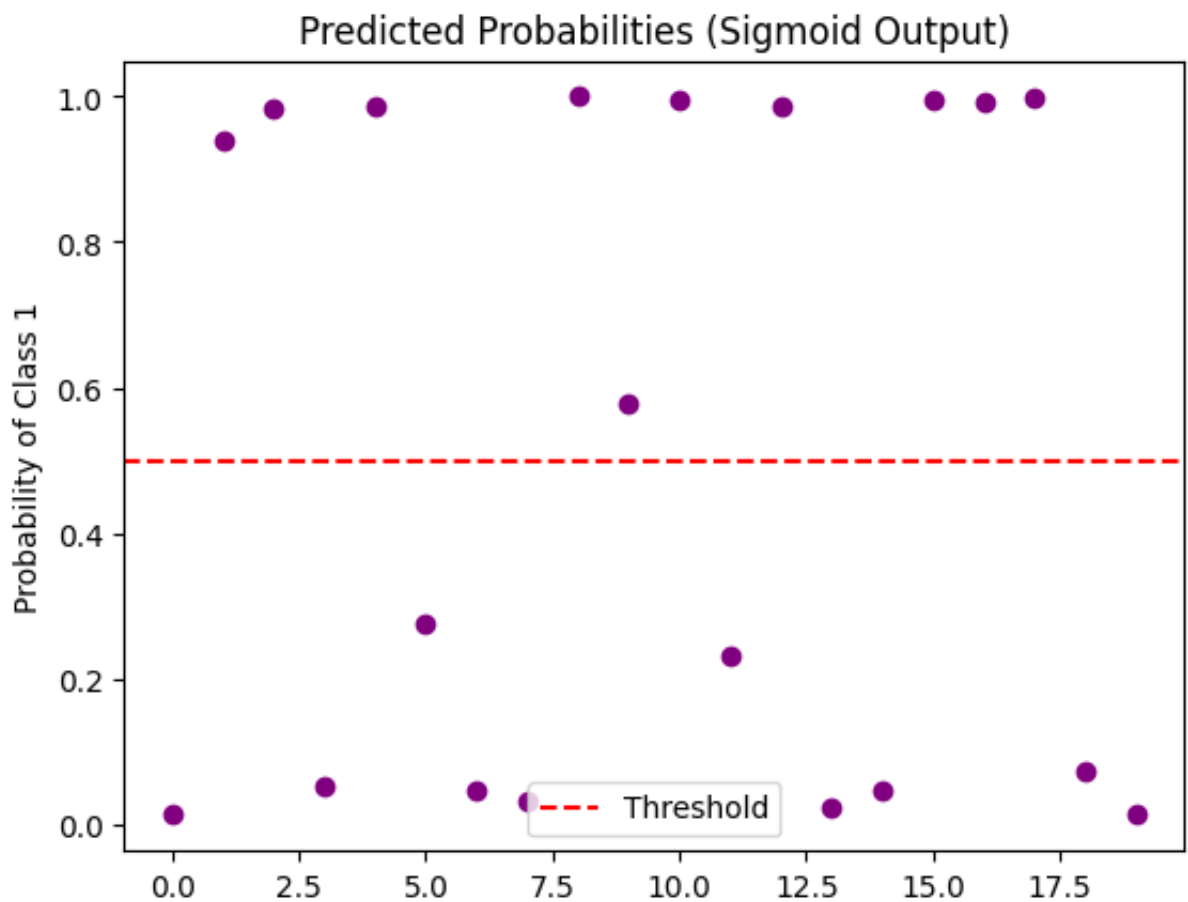
accuracy = accuracy_score(y_test, y_pred_log)
print(f"Model Accuracy: {accuracy * 100:.2f}%")
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred_log))

probabilities = log_model.predict_proba(X_test)[:, 1]
plt.scatter(range(len(probabilities)), probabilities, color='purple')
plt.axhline(y=0.5, color='r', linestyle='--', label='Threshold')
plt.title('Predicted Probabilities (Sigmoid Output)')
plt.ylabel('Probability of Class 1')
plt.legend()
plt.show()
```

Model Accuracy: 95.00%

Confusion Matrix:

```
[[10  1]
 [ 0  9]]
```



Post Lab Questions

A1 Linear Regression: This method models a linear relationship to predict a continuous dependent variable. It calculates the best-fitting ~~sto~~ straight line using equation:

$$Y = \beta_0 + \beta_1 X + E$$

Logistic Regression: This method is used specifically for binary or multinomial classification problems. It models the probability of an outcome (0 or 1) by forming an "S" shaped sigmoid curve, using equation:

$$P(Y=1) = \frac{1}{1 + e^{-(\beta_0 + \beta_1 X)}}$$

A2 Applications of Logistic Regression:

- ① Logistic regression is heavily used for classification tasks across fields like economics, biology etc.
- ② It is applied whenever you need to predict a binary outcome or probability, using a threshold value to classify data into distinct categories like 0 or 1.